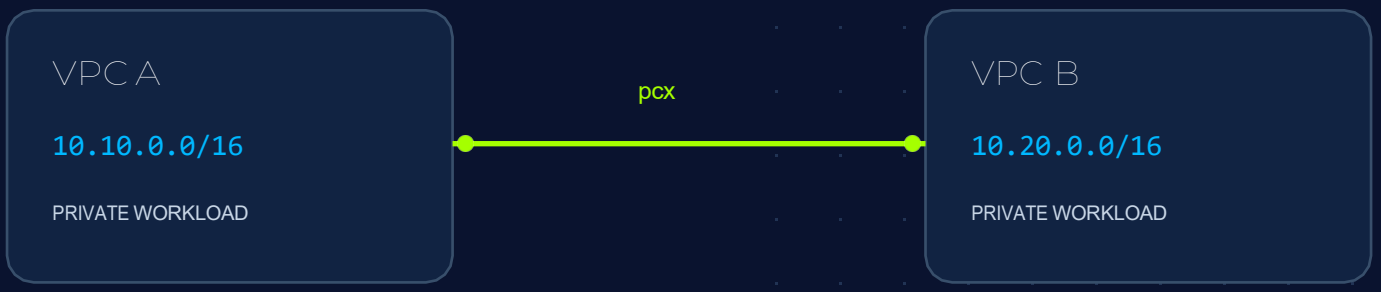
BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.

A guided networking project with EC2, custom route tables, security groups and a bastion host.



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name: **ATCHAYA. S** Batch: **1**

1. AIM

To establish a private network connection between two Amazon Virtual Private Clouds (VPCs) using VPC Peering and verify communication between EC2 instances using their private IPv4 addresses.

2. OBJECTIVES

- To create two VPCs with non-overlapping CIDR blocks.
- To create public and private subnets.
- To configure route tables and subnet associations.
- To establish and activate a VPC Peering connection.
- To configure security groups for controlled network access.
- To access the network through a Bastion host.
- To test connectivity between EC2 instances using private IPv4 addresses.
- To verify successful communication between the two VPCs.

3. PROJECT DESCRIPTION

This project demonstrates how to connect two Amazon Virtual Private Clouds using a VPC Peering connection.

VPC-A contains a Bastion host and an application server (App-A), while VPC-B contains another application server (App-B). The VPCs communicate through a private VPC Peering connection without requiring public IP addresses for communication between the application servers.

The Bastion host provides an administrative entry point for accessing the network. Route tables define the paths between the VPCs, while security groups control the permitted network traffic.

The project uses two VPCs, three EC2 instances, and one active VPC Peering connection.

Connectivity is verified by pinging App-A and App-B from the Bastion host using their private IPv4 addresses. Both tests completed successfully with 0% packet loss.

4. SYSTEM ARCHITECTURE

The project consists of the following components:

- VPC-A: 10.10.0.0/16
- VPC-B: 10.20.0.0/16
- Public subnet A: 10.10.1.0/24
- Private subnet A: 10.10.2.0/24
- Private subnet B: Located within VPC-B; its exact CIDR block should be verified in the AWS Console.
- Bastion host private IP: 10.10.1.125
- Bastion host public IP: 3.27.217.196
- App-A private IP: 10.10.2.60

- App-B private IP: 10.20.210.77
- Internet Gateway: Attached to VPC-A.
- VPC Peering connection: pcx-05c12af0684be3340 (Active).

The Bastion host is deployed in the public subnet of VPC-A and provides an administrative entry point into the network.

App-A operates within VPC-A, and App-B operates within VPC-B. Both application instances communicate using private IPv4 addresses through the VPC Peering connection.

The route tables in both VPCs contain routes to the remote VPC CIDR through the peering connection. Security groups permit the required traffic between the instances.

VPC-B does not require an Internet Gateway for communication with VPC-A through VPC Peering. VPC Peering does not automatically provide internet access to either VPC.

5. SOFTWARE AND AWS SERVICES REQUIRED

The following software and AWS services are used in this project:

- Amazon Web Services (AWS)
- Amazon Virtual Private Cloud (Amazon VPC)
- Amazon Elastic Compute Cloud (Amazon EC2)
- Internet Gateway
- VPC Peering Connection
- Route Tables
- Public and Private Subnets
- Security Groups
- Amazon Linux 2023
- EC2 Key Pair (my-key-pair)
- SSH Client
- Windows Command Prompt (CMD)
- Linux Terminal

6. IP ADDRESS PLAN

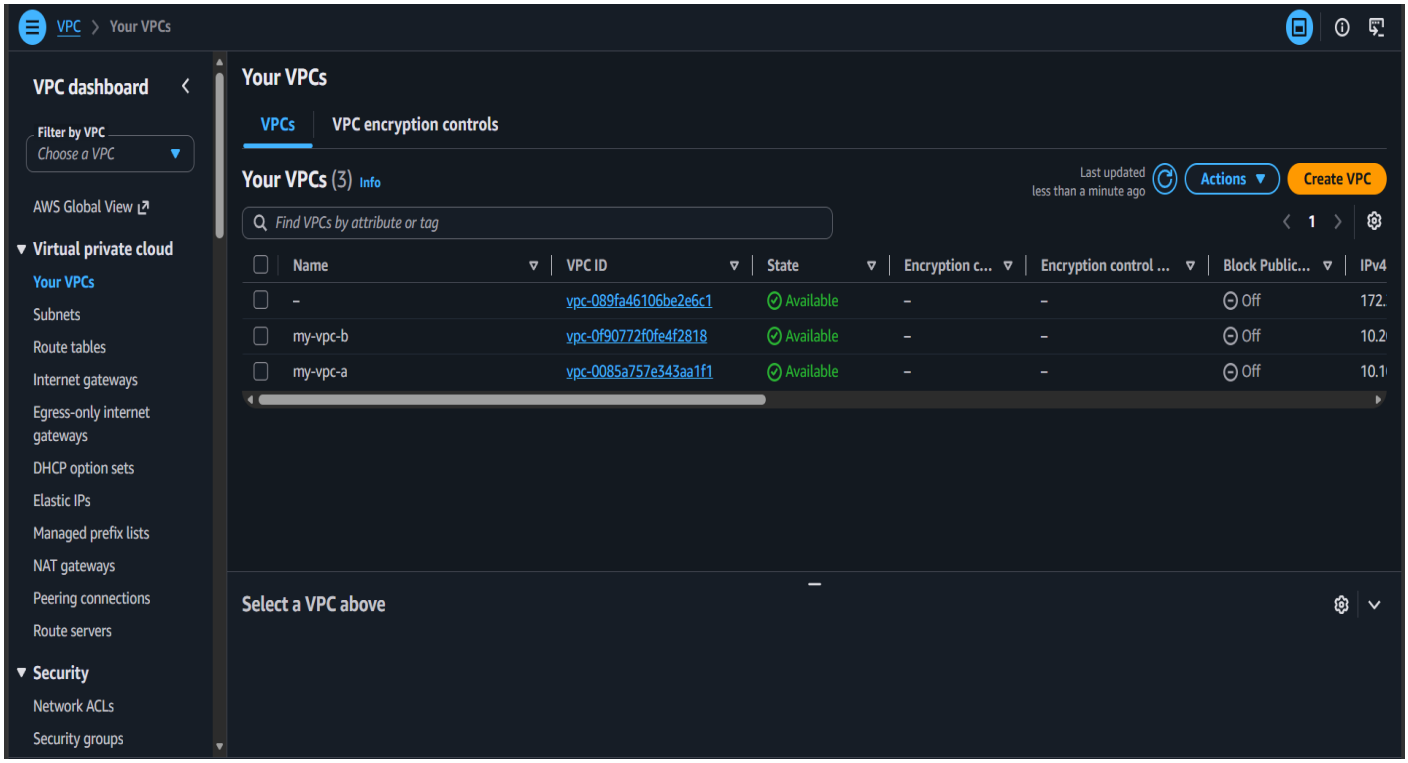
Resource	Name	CIDR / IP Address
VPC A	my-vpc-a	10.10.0.0/16
Public subnet A	my-subnet-a	10.10.1.0/24
Private subnet A	my-subnet-private	10.10.2.0/24

Resource	Name	CIDR / IP Address
VPC B	my-vpc-b	10.20.0.0/16
Private subnet B	my-vpc-private	10.20.1.0/24
Bastion host	my-bastion-1	10.10.1.10
Application server A	my-app-a	10.10.2.10
Application server B	my-app-b	10.20.1.10

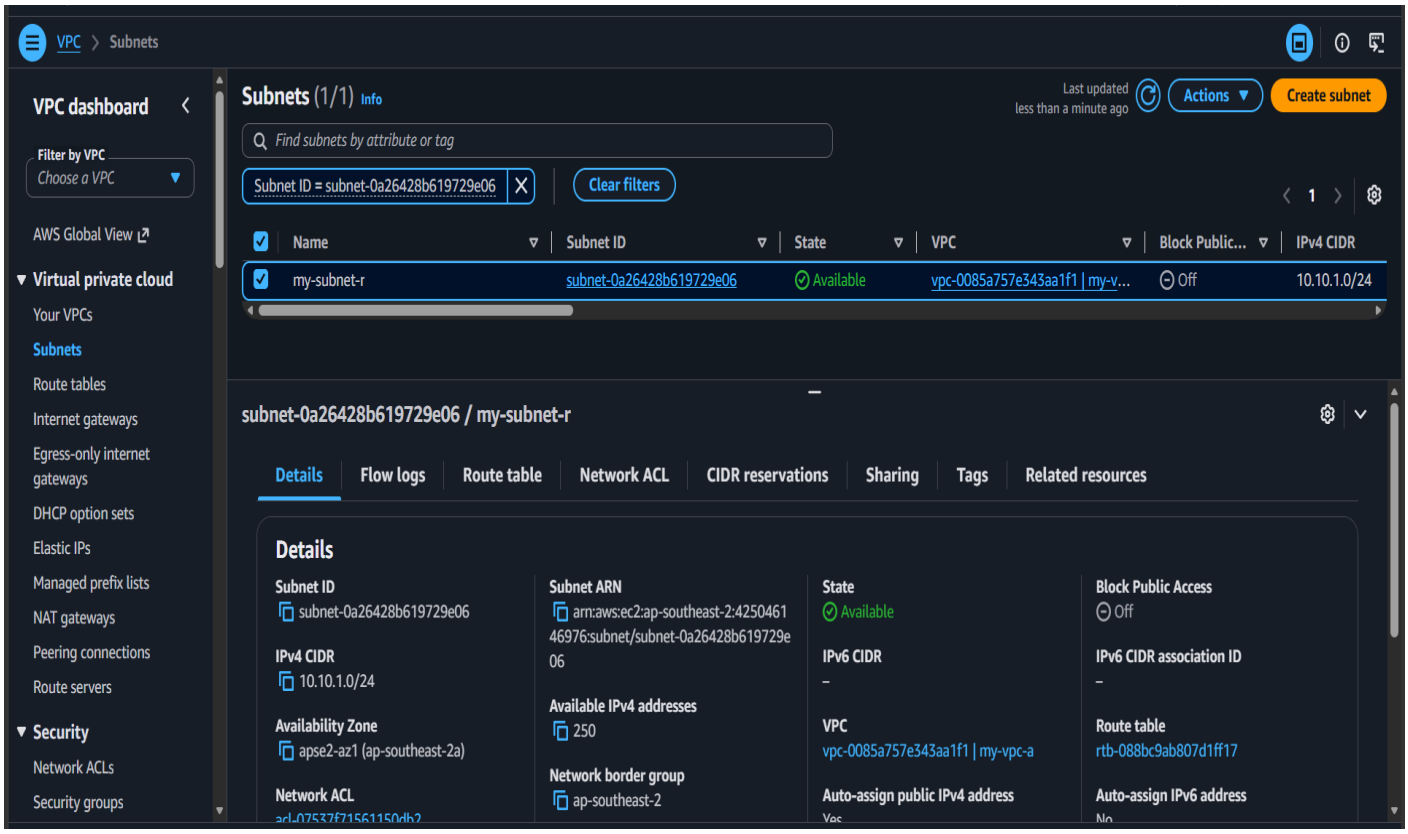
The two VPC CIDR blocks do not overlap, allowing them to be connected using VPC Peering.

7. SCREENSHOTS

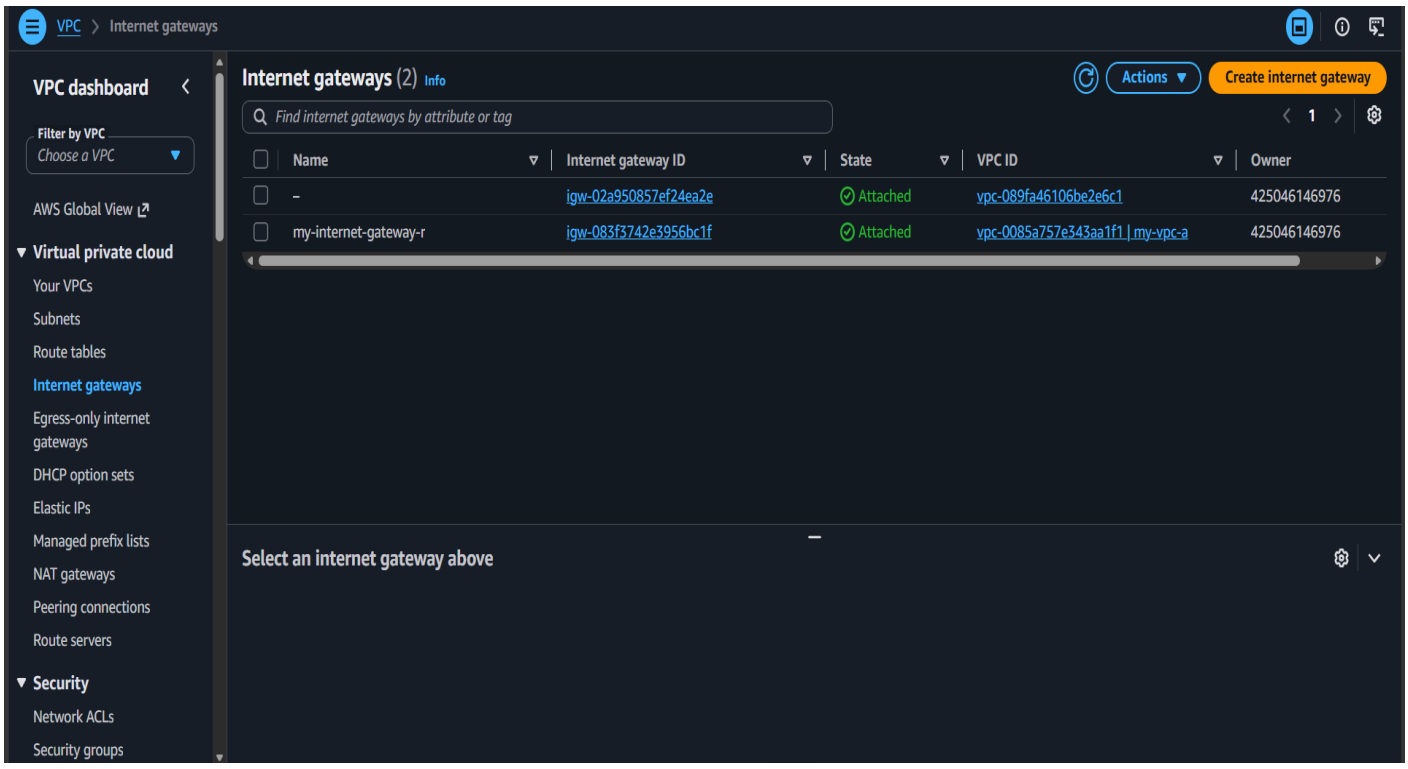
VPC's



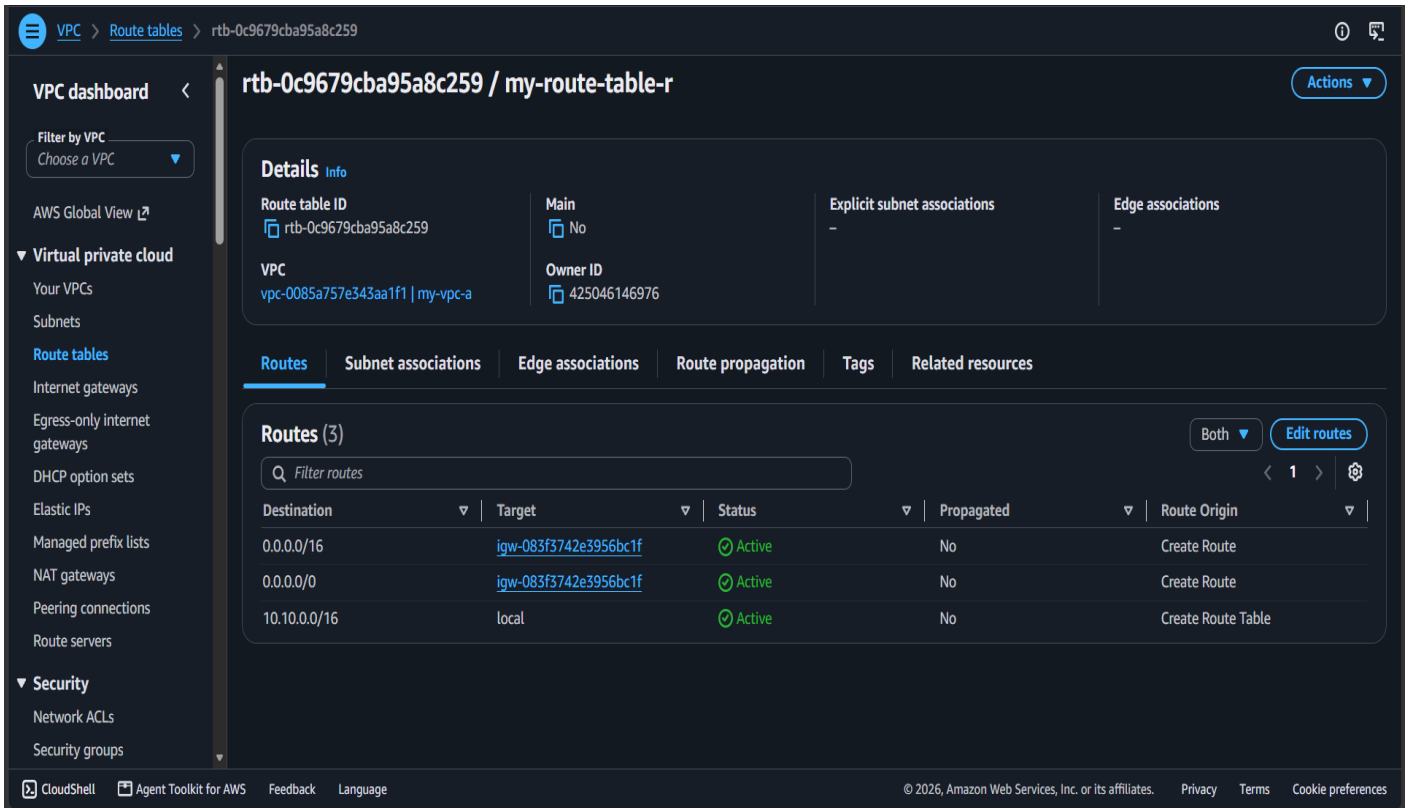
SUBNETS



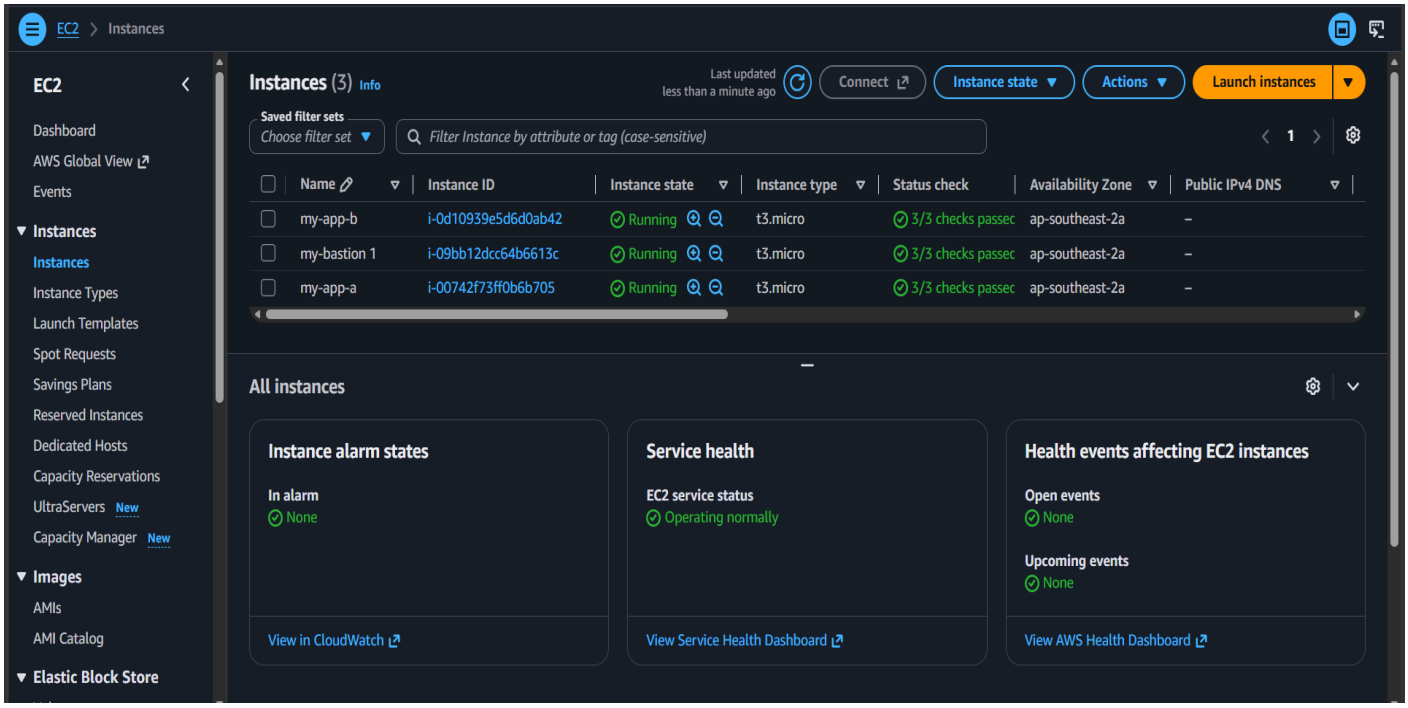
INTERNET GATE WAY



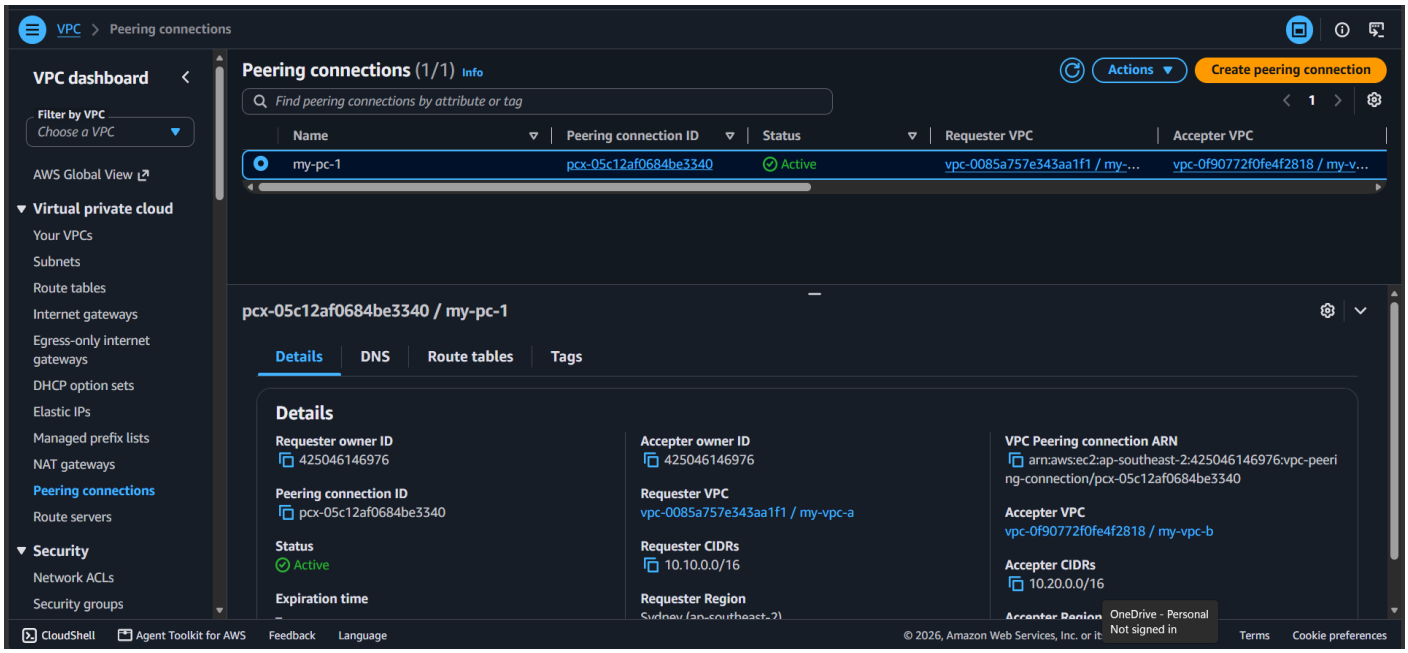
ROUTE TABLES



INSTANCES



PEERING CONNECTION



SSH connectivity

```
The authenticity of host '3.27.217.196 (3.27.217.196)' can't be established.
ED25519 key fingerprint is SHA256:H9gHdCingxO0qeReHhoQgirBZTp9Q+kj9OY5mvDy1rI.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.27.217.196' (ED25519) to the list of known hosts.
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-10-1-125 ~]$
```

Bastion - App-A connectivity

```
[ec2-user@ip-10-10-1-125 ~]$  ping -c 4 10.10.2.60
PING 10.10.2.60 (10.10.2.60) 56(84) bytes of data.
64 bytes from 10.10.2.60: icmp_seq=1 ttl=127 time=0.171 ms
64 bytes from 10.10.2.60: icmp_seq=2 ttl=127 time=0.206 ms
64 bytes from 10.10.2.60: icmp_seq=3 ttl=127 time=0.200 ms
64 bytes from 10.10.2.60: icmp_seq=4 ttl=127 time=0.206 ms

--- 10.10.2.60 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3097ms
rtt min/avg/max/mdev = 0.171/0.195/0.206/0.014 ms
[ec2-user@ip-10-10-1-125 ~]$
```

Bastion - App-B connectivity

```
       _/m/'
Last login: Tue Oct 27 18:01:45 2026 from 27.60.165.111
[ec2-user@ip-10-10-1-125 ~]$ ping -c 4 10.20.210.77
PING 10.20.210.77 (10.20.210.77) 56(84) bytes of data.
64 bytes from 10.20.210.77: icmp_seq=1 ttl=127 time=0.213 ms
64 bytes from 10.20.210.77: icmp_seq=2 ttl=127 time=0.207 ms
64 bytes from 10.20.210.77: icmp_seq=3 ttl=127 time=0.182 ms
64 bytes from 10.20.210.77: icmp_seq=4 ttl=127 time=0.174 ms

--- 10.20.210.77 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3153ms
rtt min/avg/max/mdev = 0.174/0.194/0.213/0.016 ms
[ec2-user@ip-10-10-1-125 ~]$ ping -c 4 10.10.2.60
PING 10.10.2.60 (10.10.2.60) 56(84) bytes of data.

--- 10.10.2.60 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3118ms
```

Configure Security Groups

Create the following security groups and inbound rules.

Security Group	Type / Port	Source
my-bastion-1	SSH / TCP 22	Your current public IPv4 address /32
my-app-a	SSH / TCP 22	10.10.1.10/32
my-app-a	ICMPv4 Echo Request	10.20.1.10/32
my-app-b	SSH / TCP 22	10.10.1.10/32
my-app-b	ICMPv4 Echo Request	10.10.2.10/32

Keep the default outbound allow-all rules for this lab. Do not expose SSH or ICMP to 0.0.0.0/0.

8. TROUBLESHOOTING

8.1 Bastion SSH Connection Times Out

- Check the current public IPv4 address of the Bastion host.
- Verify that the security group allows SSH traffic (TCP port 22) from the client's current public IP address.
- Confirm that the Bastion host is deployed in the public subnet.
- Verify that the public subnet route table has a default route (0.0.0.0/0) to the Internet Gateway.

8.2 App-A SSH Connection Fails

- Verify App-A's private IPv4 address: 10.10.2.60.
- Confirm that the security group allows SSH (TCP port 22) from the Bastion host's private IP, 10.10.1.125/32.
- Verify that the correct EC2 key pair is used.
- Use the ec2-user username for Amazon Linux 2023.
- Confirm that the route table and network access settings permit the connection.

8.3 App-B SSH Connection Fails

- Verify App-B's private IPv4 address: 10.20.210.77.
- Check that VPC-B's route table contains a route to 10.10.0.0/16 through the active VPC Peering connection.
- Confirm that VPC-A's relevant route table contains a route to 10.20.0.0/16 through the same peering connection.
- Confirm that App-B's security group allows SSH (TCP port 22) from the Bastion host's private IP, 10.10.1.125/32.
- Verify the correct SSH key pair and username.

8.4 Ping Fails After VPC Peering Becomes Active

- Verify the route tables associated with the source and destination subnets.
- Confirm that the VPC Peering connection is active.
- Check the destination instance's security group for an appropriate ICMP inbound rule.
- Verify the actual private IPv4 addresses of the instances.
- Confirm that the relevant network routes allow return traffic.

8.5 VPC Peering Connection Creation Fails

- Check that the VPC CIDR blocks do not overlap.
- Verify that the correct VPCs and AWS Region are selected.
- Check the peering connection's status and accept the request if required.

8.6 SSH Reports Permission denied (publickey)

- Verify that the correct private key file is being used.
- Confirm that the key pair matches the one assigned to the EC2 instance.
- Check the SSH username (ec2-user for Amazon Linux 2023).
- Verify that the key file path is correct.

9. CLEANUP

After capturing the required screenshots and saving the project report, perform the following cleanup steps if you no longer need the AWS resources:

1. Terminate the project EC2 instances: my-bastion 1, my-app-a, and my-app-b.
2. Check for unused EBS volumes and snapshots, and remove them if they are no longer required.
3. Release any unused Elastic IP addresses created for the project.
4. Delete the unused EC2 key pair if it is no longer required.
5. Remove the custom peering routes and delete the VPC Peering connection when it is no longer needed.
6. Delete project security groups after they are no longer associated with network interfaces or other resources.
7. Delete the project subnets and custom route tables.
8. Detach and delete the Internet Gateway associated with VPC-A.
9. Delete VPC-A and VPC-B.
10. Review the AWS account for any remaining project resources that could incur charges.
11. Save screenshots or other evidence of the completed cleanup.

.

10. CONCLUSION

The AWS VPC Peering project demonstrates how two Amazon Virtual Private Clouds with non-overlapping CIDR blocks can communicate using private IPv4 addresses.

The project uses two VPCs, three EC2 instances, route tables, security groups, and a VPC Peering connection to establish controlled network communication. A Bastion host provides an administrative entry point for accessing the instances.

Connectivity testing was successful: the Bastion host communicated with App-A (10.10.2.60) and App-B (10.20.210.77), with both ping tests reporting 0% packet loss.

These results demonstrate successful connectivity from the Bastion host to both application instances, including communication across the VPC Peering connection.

The project report should include actual AWS Console screenshots and terminal outputs as evidence. Reverse-direction ping tests, private SSH access to both application instances, and controlled failure-and-recovery tests should be reported as completed only if they were actually performed and verified.